

نام خداوند بخشنده مهربان



Self Adaptive Systems

Professor: Mehran Alidoostnia

Author: Abolfazl Omidian

Major : IT – Enterprise Architecture

Student Number: 404443019

Dynamic Digital Twin Architecture for Intelligent Resource Management in Crisis Situations

1. Introduction and Problem Statement

In modern software engineering, resource management within cyber-physical systems faces severe challenges. In traditional architectures, system modeling is executed in a completely static manner; that is, the system designer assumes rigid, predefined parameters during the design phase and implements the architecture accordingly. However, the physical world around us—especially in critical environments such as healthcare facilities—is highly dynamic, stochastic, and unpredictable. Sudden, high-volume influxes of users during crises impose massive fluctuations on the system.

The fundamental issue arises when the physical reality changes, but the system's digital model remains incapable of perceiving these modifications due to its static nature. This phenomenon leads to an architectural blind spot. In such scenarios, the software layer loses its capacity for resource orchestration, causing a severe drop in the Quality of Service. Consequently, crisis management must inadvertently be handled manually by human operators using makeshift workarounds, which exponentially increases the risk of human error. To mitigate this issue, this report evaluates a novel architecture based on self-adaptive systems.

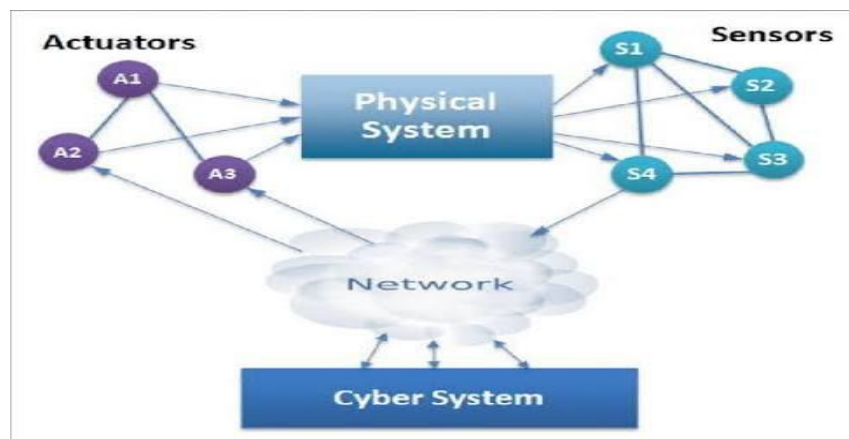


Fig1: Cyber physical systems architecture diagram

2. Paradigm Shift Toward Self-Adaptive Systems

Overcoming the aforementioned challenges requires a significant paradigm shift at the architectural layer. This shift in perspective is achieved by blending enterprise architecture concepts with intelligent control mechanisms, such as a self-adaptive control loop. The foundational differences between this novel approach and traditional frameworks are centered around three main axes:

- **Architectural Structure:** In legacy systems, structural orchestration was completely fixed, and modifications were applied offline. In the proposed architecture, structural synchronization is executed at runtime via a semantic reflection mechanism.
- **Demand Management:** The resource allocation process has shifted from rigid scheduling policies to a fully declarative structure, enabling the platform to reconfigure itself based on the incoming load's lifecycle.
- **Crisis Decision-Making:** Crisis management is offloaded from the human operator and assigned to constraint-based optimization algorithms, allowing the machine to compute the most cost-effective path using mathematical models.

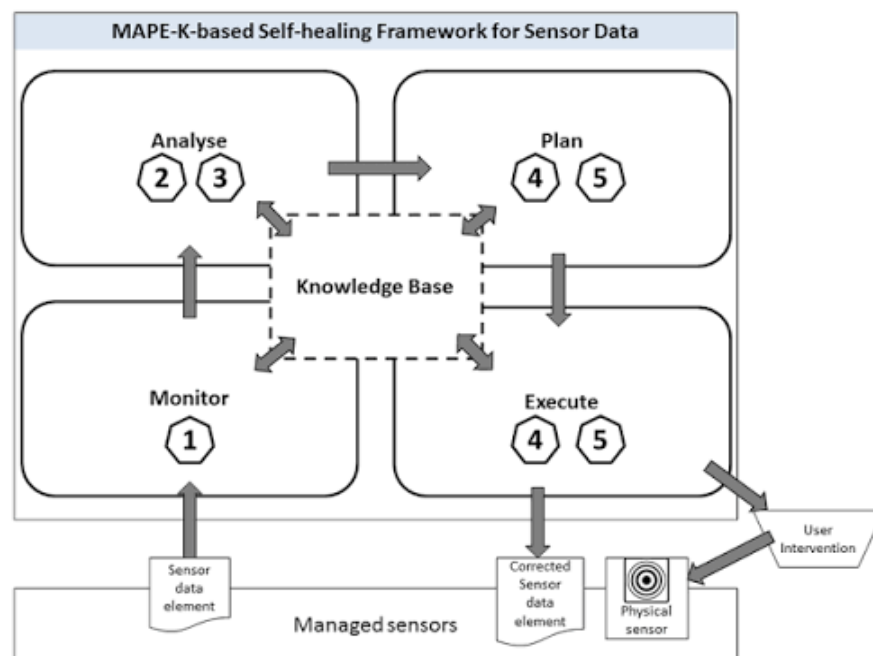


Fig2: MAPE-K loop control architecture

3. Description of the Overall System Architecture

The proposed architecture relies on three primary pillars that collectively establish a closed-loop feedback control layer over the physical environment:

Pillar One: Runtime Knowledge Graph

This component resides within the digital twin layer and is responsible for environmental perception. Any modification in the physical layer is rapidly reflected within this semantic graph.

Pillar Two: Lifecycle Status Management

This section functions as a monitoring and triggering engine. Its core task is to continuously track vital system metrics, such as occupancy rates and incoming traffic, to issue resource migration signals whenever system stability is jeopardized.

Pillar Three: Decision-Making and Optimization Engine

This component is responsible for solving mathematical equations to strike an optimal balance between system resilience and operational costs. To ensure operational safety, the output of this engine is passed to a final supervisor using a human-in-the-loop pattern. Upon validation, the self-reconfiguration process is executed directly on the environment.

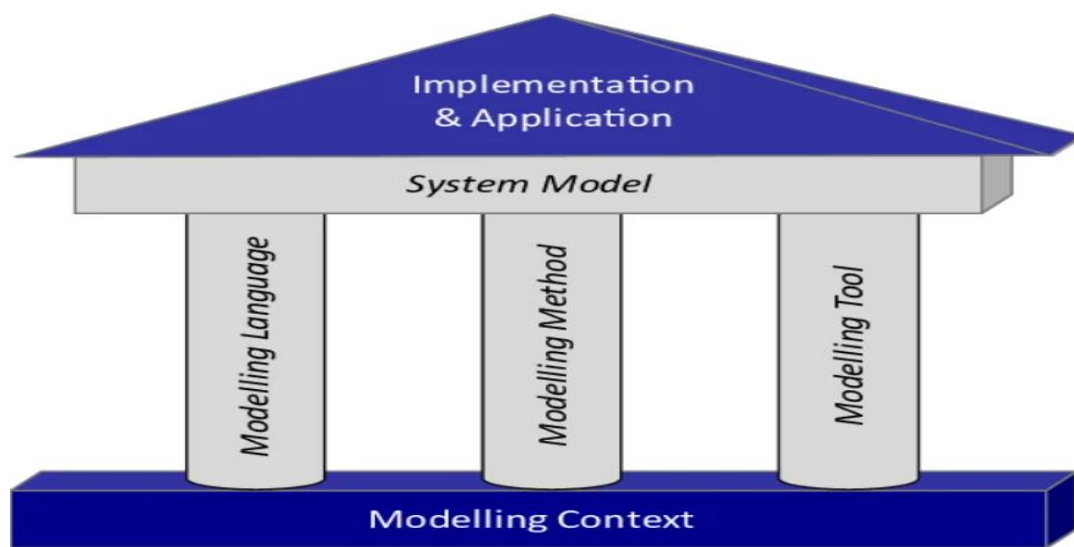


Fig3: System architecture three pillars diagram

4. Semantic Reflection Layer and Lifecycle Management

To comprehend environmental dynamics, the system models all physical components—such as rooms, personnel, and capacities—as nodes, and their interrelations as edges within a semantic graph. As new data arrives from the environment, the runtime reflection process triggers, updating the graph through node additions, modifications, or deprecations.

A key concept within this layer is passive resources. Resources like corridors or administrative spaces, which should not be utilized under normal conditions, are designated with a heavy penalty weight in the graph. This setup ensures that the system triggers them only as a last resort during severe crises.

The lifecycle manager supervises the system status, with its critical threshold calibrated at ninety percent of total capacity. The system utilizes the following mathematical formula to calculate the allowable intake ceiling at runtime:

As soon as the system load breaches this Gamma threshold, a state-change signal

$$\Gamma = \lfloor \frac{\Omega \cdot \epsilon}{100} \rfloor$$

is broadcast, activating the orchestrator to discover a new, optimized configuration.

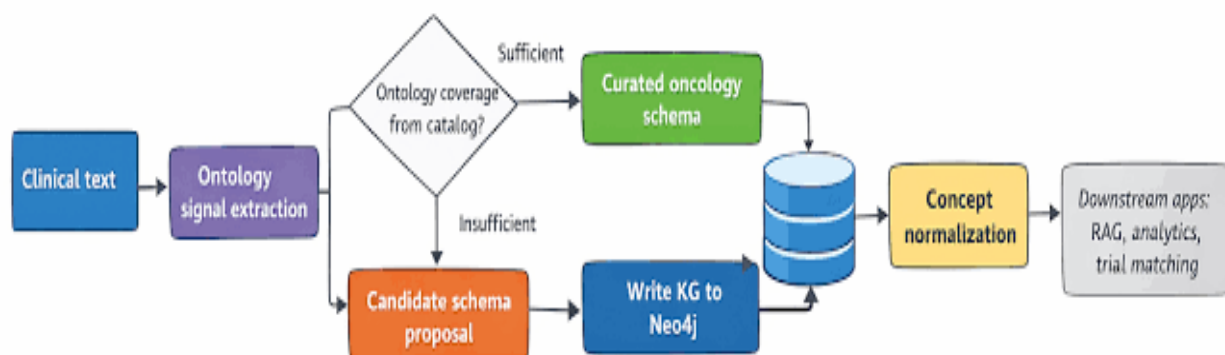


Fig4: Dynamic ontology graph nodes edges

5. Mathematical Decision-Making Model and Penalty-Based Optimization

During a crisis, the decision engine must identify the lowest-cost and safest configuration. This process is executed through the following optimization formula:

$$\min \sum (\delta_r \cdot a_{id,r}) + \sigma_{id}$$

This mathematical model ensures that the system selects configurations yielding the minimum total penalty weights. For instance, repurposing an administrative office incurs a lower penalty compared to using a corridor, which carries high health and safety risks. Furthermore, the Sigma variable in this formula guarantees that the relocation and movement of existing users are kept to an absolute minimum to prevent system disruption and operational overhead.

6. Evaluation Environment and Simulation Scenarios

To validate the efficacy of this architecture, a simulation-based proof-of-concept environment was established to measure the system's runtime behavior. The primary objective was to observe whether the system could convert a stochastic and chaotic influx of inputs into a highly optimized resource allocation pattern. The base capacity of the system in this experiment was set to 45 units.

To introduce a realistic challenge, the base input and arrival rates were modeled using a Poisson distribution. Then, to conduct a stress test, the arrival rate was spiked with a massive multiplier during a specific timeframe. This simulated a sudden surge or crisis condition, allowing us to evaluate the reaction speed of the self-adaptive mechanisms.

7. Data Analysis and Experimental Findings

The stress test results demonstrated that as soon as the incoming workload breached the allowable threshold, the system expanded its physical capacity in a highly localized and calculated manner. The system initially brought low-risk spaces online and strictly avoided high-penalty, high-risk resources unless absolutely forced by the constraints.

The long-term experimental output graph verified the exceptional elasticity of this architecture; the system maintained an elevated capacity throughout the duration of the crisis and promptly deallocated additional resources to return to the baseline state once the workload subsided, preventing long-term resource wastage.

Naturally, this intelligence comes with an architectural trade-off. Utilizing semantic reflection introduces a computational overhead during periods of extreme environmental volatility. However, the experiments proved that this processing load is strictly dependent on environmental fluctuation; as soon as the system stabilizes in its crisis state, continuous graph updates halt, and the computational cost drops by over ninety percent.

8. Conclusion and Future Horizons

This research demonstrates how qualitative concepts and human protocols can be translated into the mathematical language of machines via penalty weights, ensuring that the system always yields decisions aligned with designated organizational standards.

The tripartite framework designed in this work (comprising semantic reflection, lifecycle management, and penalty-based optimization) offers a comprehensive blueprint. Far beyond healthcare environments, this architecture can be generalized and implemented across various dynamic domains, including cloud infrastructure provisioning, smart energy distribution grids, and supply chain management.